ІНФОРМАЦІЙНЕ ЗАБЕЗПЕЧЕННЯ
ПРОМИСЛОВИХ РОБОТІВ

Комп’ютерний практикум No6

Тема роботи: Розроблення розподіленої системи керування робототехнічним комплексом.
Мета роботи: Набути практичних навичок проєктування та реалізації розподіленої системи керування робототехнічним комплексом. Навчитися організовувати ієрархічну взаємодію локальних контролерів і координатора, обмін даними за промисловим протоколом, інтеграцію додаткових підсистем, журналювання подій та обробку відмов обладнання і зв’язку.

Постановка задачі:

1. Сформулювати задачу керування робототехнічним комплексом. Визначити призначення комплексу, склад обладнання та загальну послідовність його роботи. За основу можна використати задачу й результати попереднього практикуму.

2. Розробити архітектуру розподіленої системи керування. Передбачити щонайменше два локальні контролери різних пристроїв і координатор комплексу. Визначити їхні функції, межі відповідальності та рівні ієрархії. Побудувати структурну схему системи із зазначенням зв’язків між компонентами.

3. Описати логіку роботи контролерів і координатора. Визначити стани, події, умови переходів, команди, часові обмеження та аварійні реакції. Побудувати відповідні діаграми станів і таблиці переходів. Використати або адаптувати FSM, реалізований у попередньому практикумі.

4. Вибрати промисловий протокол і визначити інтерфейси обміну. Використати щонайменше один протокол, наприклад Modbus TCP або OPC UA. Описати передавання команд, параметрів, станів і кодів помилок. Передбачити ідентифікацію операцій, підтвердження прийняття та виконання команд, а також контроль працездатності зв’язку.

5. Реалізувати розподілене керування мовою C++ або Python. Організувати виконання локальних контролерів і координатора в окремих процесах на одному або кількох комп’ютерах. Забезпечити обмін через обраний протокол, використовуючи готові бібліотеки. Підключити контролери до

обладнання в симуляторі або його тестових моделей.

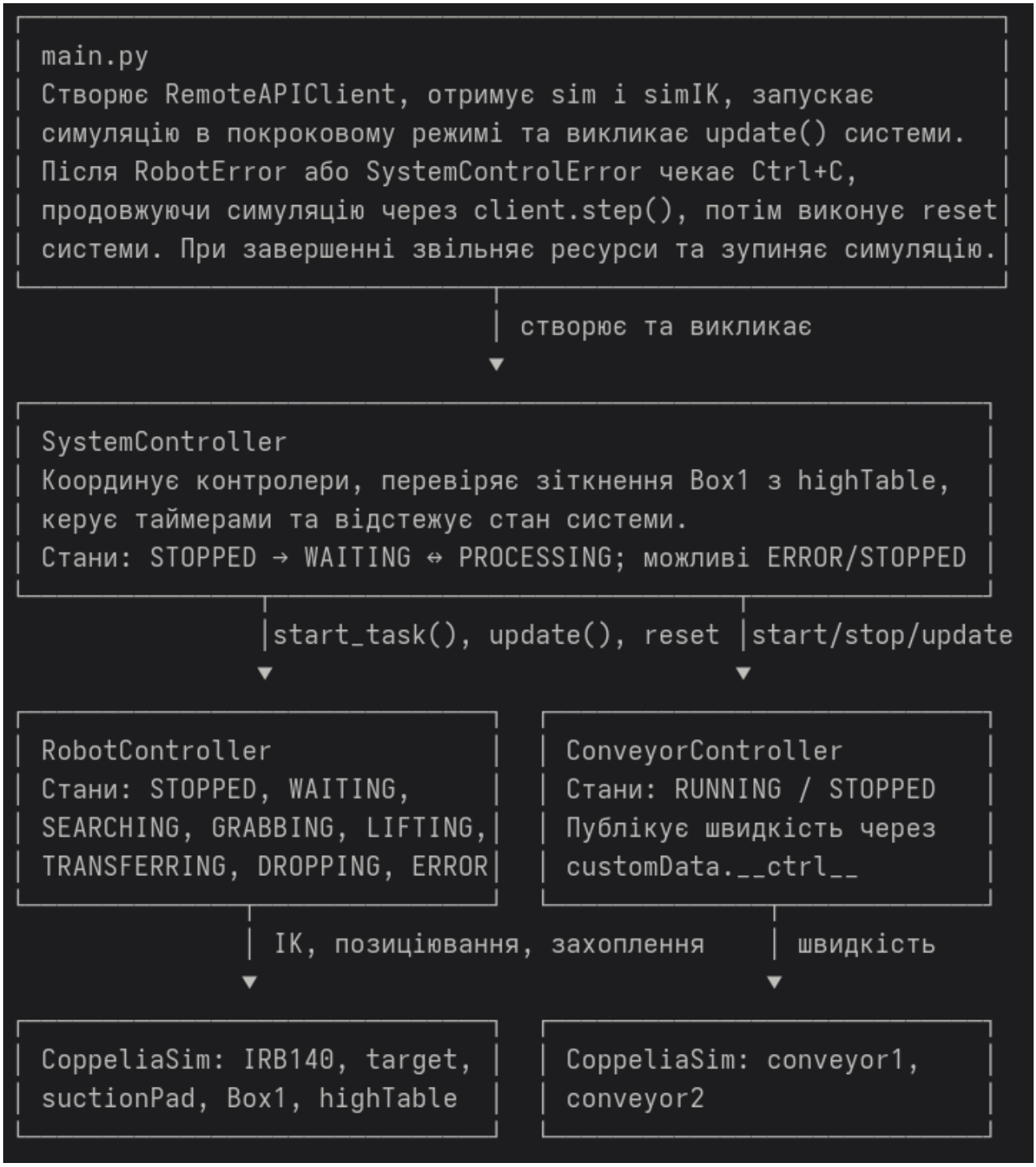
6. Розробити та підключити додаткову підсистему за вибором. Наприклад: розпізнавання об’єктів за зображенням камери, керування буфером накопичення, розподіл завдань між роботами або інтерфейс оператора. Визначити її вхідні й вихідні дані та використання результатів у роботі комплексу.

7. Забезпечити моніторинг і журналювання роботи системи. Реєструвати команди, штатні й аварійні події, зміни станів, втрату та відновлення зв’язку. Доповнити записи часом, ідентифікатором компонента та номером операції, щоб можна було відтворити взаємодію контролерів.

8. Провести інтеграційне тестування комплексу. Перевірити повний штатний цикл, відмову окремого пристрою, втрату зв’язку, затримку підтвердження та повторне надходження тієї самої команди. Переконатися, що повторне отримання команди не спричиняє повторного виконання вже прийнятої операції. Перевірити скидання аварій і повторний запуск після усунення несправностей.

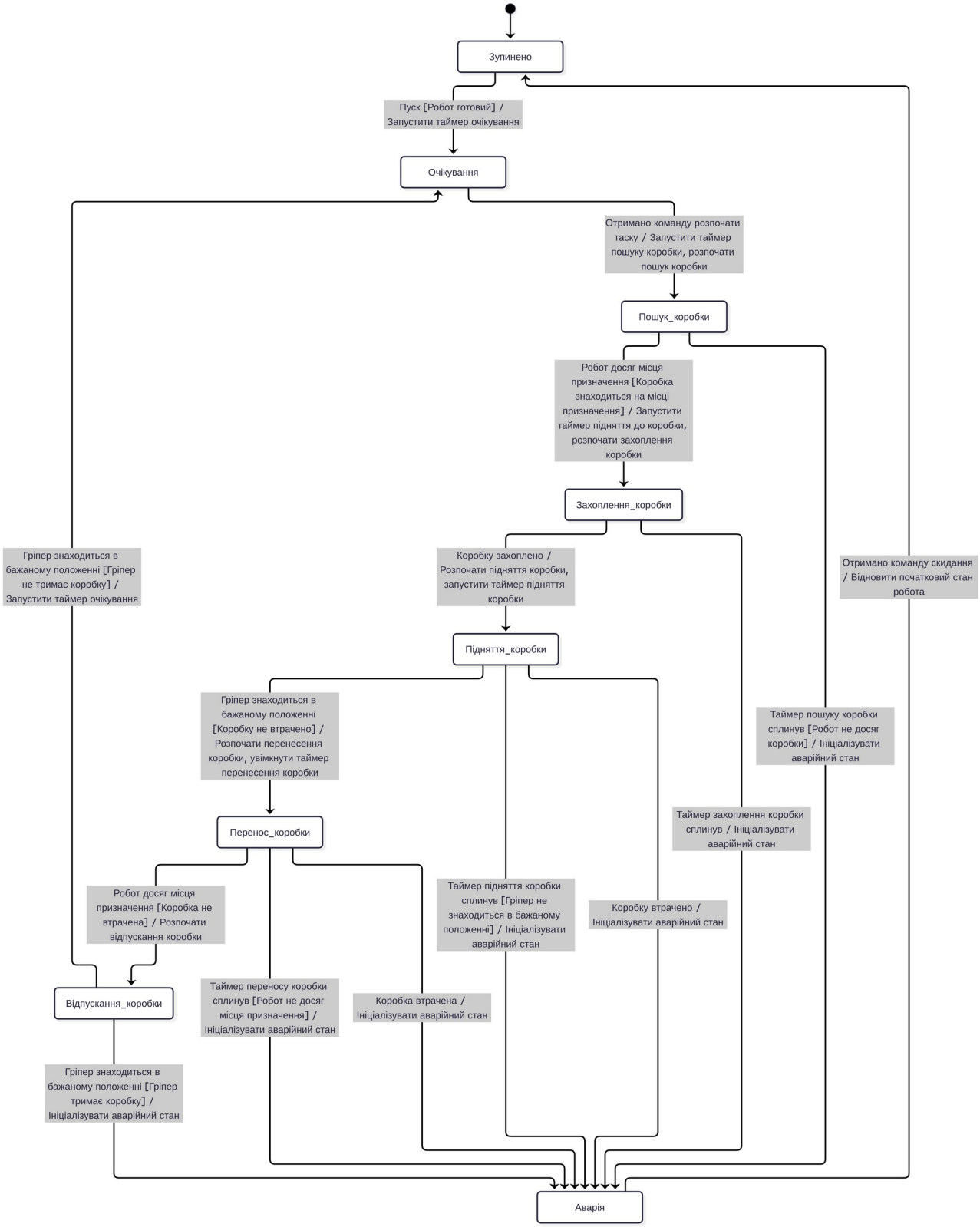
9. Оформити результати роботи та інструкцію запуску. Подати постановку задачі, структурну схему, діаграми станів, опис інтерфейсів обміну, програмну реалізацію, тестові сценарії, фрагменти журналів і висновки. Зазначити порядок запуску компонентів та необхідні налаштування з’єднань.

Структурна схема:

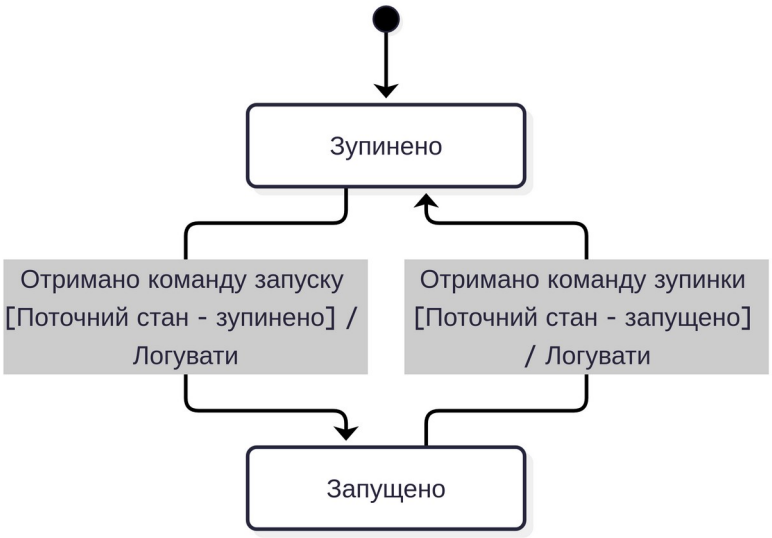


Діаграми станів:

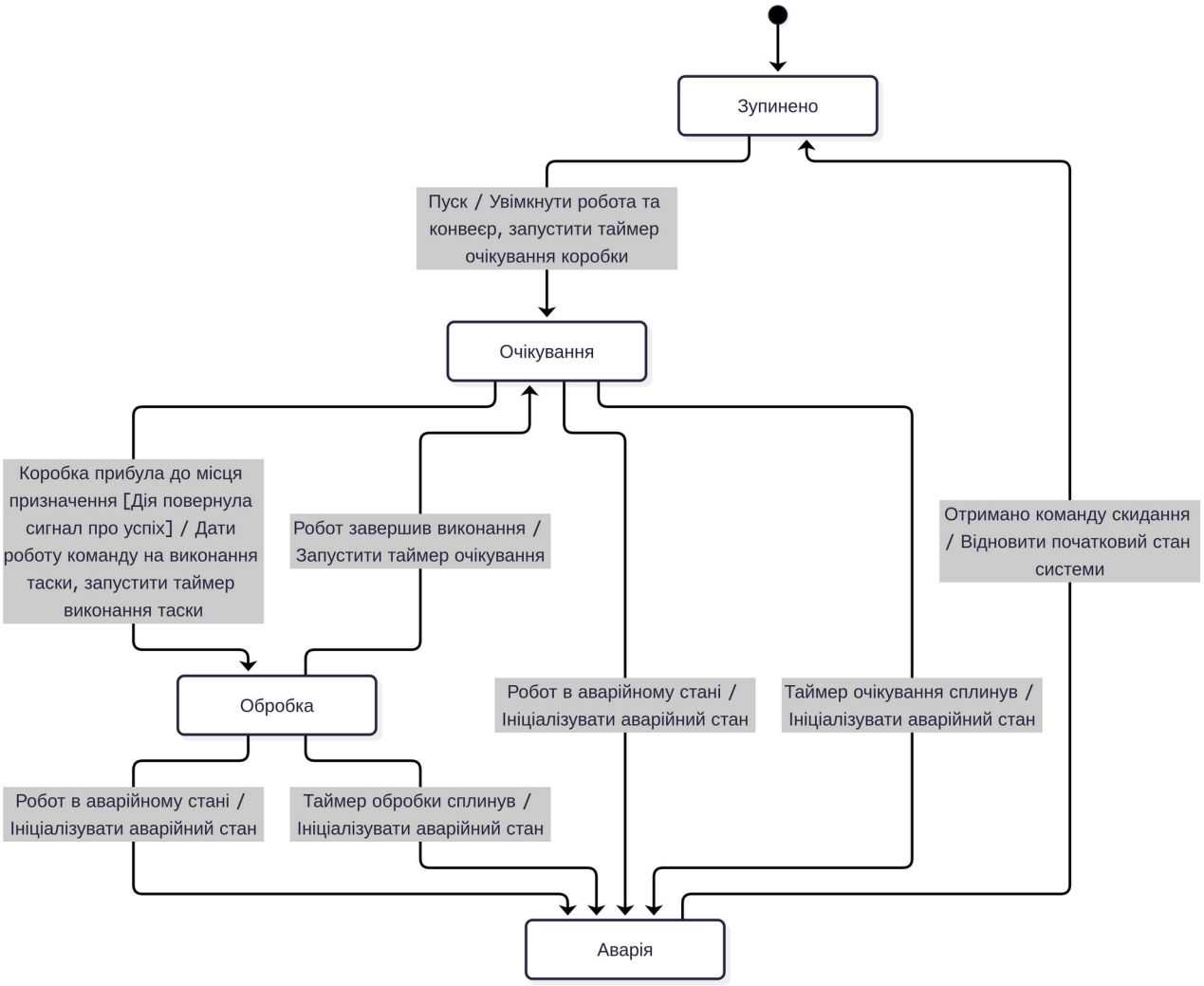
Робот:



Конвеєр:



Системна:



Код програми:

[Посилання на гітхаб]

Приклад тестових сценаріїв:
1. Робота в штатному режими.
2. Робота в штатному режимі, після чого коробка прибирається з конвеєра в довільний момент.
3. Робота в штатному режимі, після чого коробка прибирається з присоски в довільний момент.
4. Відновлення штатної роботи після сценаріїв 2 або 3.
5. Перевірка коректності завершення сцени за сигналом ctr+c.

Демонстрація журналу для сценаріїв 2+4+5

```
(.venv)
[nix-shell:~/Documents/5sem/robots/cw6]$ python main.py
[0001] [2026-11-11 02:38:39.093] [RobotController] [STOPPED] SYSTEM: Robot controller initialized
[0002] [2026-11-11 02:38:39.094] [SystemController] [STOPPED] COMMAND: Start command received
[0003] [2026-11-11 02:38:39.094] [RobotController] [STOPPED] COMMAND: Start command received
[0004] [2026-11-11 02:38:39.094] [RobotController] [WAITING] STATE_CHANGE: Robot state changed | STOPPED -> WAITING | Reason: Start command
[0005] [2026-11-11 02:38:39.095] [SystemController] [WAITING] STATE_CHANGE: System state changed | STOPPED -> WAITING | Reason: System started
[0006] [2026-11-11 02:38:46.088] [RobotController] [WAITING] COMMAND: Box processing task received
[0007] [2026-11-11 02:38:46.088] [RobotController] [SEARCHING] STATE_CHANGE: Robot state changed | WAITING -> SEARCHING | Reason: Task started by system controller
[0008] [2026-11-11 02:38:46.088] [ConveyorController] [STOPPED] COMMAND: Conveyor stopped
[0009] [2026-11-11 02:38:46.088] [ConveyorController] [STOPPED] STATE_CHANGE: Conveyor state changed | RUNNING -> STOPPED | Reason: Stop command
[0010] [2026-11-11 02:38:46.089] [SystemController] [PROCESSING] STATE_CHANGE: System state changed | WAITING -> PROCESSING | Reason: Box processing task started
[0011] [2026-11-11 02:38:47.138] [RobotController] [GRABBING] STATE_CHANGE: Robot state changed | SEARCHING -> GRABBING | Reason: Robot aimed at box, grabbing state activated
[0012] [2026-11-11 02:38:47.138] [RobotController] [GRABBING] NORMAL: Attempting to grab box
[0013] [2026-11-11 02:38:48.653] [RobotController] [GRABBING] NORMAL: Box successfully grabbed
[0014] [2026-11-11 02:38:48.653] [RobotController] [LIFTING] STATE_CHANGE: Robot state changed | GRABBING -> LIFTING | Reason: Box successfully grabbed
[0015] [2026-11-11 02:38:48.653] [RobotController] [LIFTING] NORMAL: Lifting box
[0016] [2026-11-11 02:38:48.689] [ConveyorController] [RUNNING] COMMAND: Conveyor started
[0017] [2026-11-11 02:38:48.689] [ConveyorController] [RUNNING] STATE_CHANGE: Conveyor state changed | STOPPED -> RUNNING | Reason: Start command
[0018] [2026-11-11 02:38:48.946] [RobotController] [LIFTING] NORMAL: Box lifted successfully
[0019] [2026-11-11 02:38:48.947] [RobotController] [TRANSFERRING] STATE_CHANGE: Robot state changed | LIFTING -> TRANSFERRING | Reason: Box lifted successfully
[0020] [2026-11-11 02:38:48.947] [RobotController] [TRANSFERRING] NORMAL: Transferring box to destination
[0021] [2026-11-11 02:38:49.401] [RobotController] [TRANSFERRING] NORMAL: Destination
```

```
reached; releasing box
[0022] [2026-11-11 02:38:49.401] [RobotController] [DROPPING] STATE_CHANGE: Robot state changed | TRANSFERRING -> DROPPING | Reason: Robot reached the destination
[0023] [2026-11-11 02:38:49.445] [RobotController] [DROPPING] NORMAL: Box successfully dropped
[0024] [2026-11-11 02:38:49.446] [RobotController] [WAITING] STATE_CHANGE: Robot state changed | DROPPING -> WAITING | Reason: Box released successfully
[0025] [2026-11-11 02:38:49.446] [RobotController] [WAITING] NORMAL: Waiting for next task
[0026] [2026-11-11 02:38:49.482] [SystemController] [WAITING] STATE_CHANGE: System state changed | PROCESSING -> WAITING | Reason: Box processing completed
[0027] [2026-11-11 02:38:52.942] [RobotController] [WAITING] COMMAND: Box processing task received
[0028] [2026-11-11 02:38:52.943] [RobotController] [SEARCHING] STATE_CHANGE: Robot state changed | WAITING -> SEARCHING | Reason: Task started by system controller
[0029] [2026-11-11 02:38:52.943] [ConveyorController] [STOPPED] COMMAND: Conveyor stopped
[0030] [2026-11-11 02:38:52.943] [ConveyorController] [STOPPED] STATE_CHANGE: Conveyor state changed | RUNNING -> STOPPED | Reason: Stop command
[0031] [2026-11-11 02:38:52.943] [SystemController] [PROCESSING] STATE_CHANGE: System state changed | WAITING -> PROCESSING | Reason: Box processing task started
[0032] [2026-11-11 02:38:53.134] [RobotController] [GRABBING] STATE_CHANGE: Robot state changed | SEARCHING -> GRABBING | Reason: Robot aimed at box, grabbing state activated
[0033] [2026-11-11 02:38:53.134] [RobotController] [GRABBING] NORMAL: Attempting to grab box
[0034] [2026-11-11 02:38:53.148] [ConveyorController] [RUNNING] COMMAND: Conveyor started
[0035] [2026-11-11 02:38:53.148] [ConveyorController] [RUNNING] STATE_CHANGE: Conveyor state changed | STOPPED -> RUNNING | Reason: Start command
[0036] [2026-11-11 02:38:54.258] [RobotController] [GRABBING] NORMAL: Box successfully grabbed
[0037] [2026-11-11 02:38:54.258] [RobotController] [LIFTING] STATE_CHANGE: Robot state changed | GRABBING -> LIFTING | Reason: Box successfully grabbed
[0038] [2026-11-11 02:38:54.258] [RobotController] [LIFTING] NORMAL: Lifting box
[0039] [2026-11-11 02:38:54.492] [RobotController] [LIFTING] NORMAL: Box lifted successfully
[0040] [2026-11-11 02:38:54.493] [RobotController] [TRANSFERRING] STATE_CHANGE: Robot state changed | LIFTING -> TRANSFERRING | Reason: Box lifted successfully
[0041] [2026-11-11 02:38:54.493] [RobotController] [TRANSFERRING] NORMAL: Transferring box to destination
[0042] [2026-11-11 02:38:54.858] [RobotController] [TRANSFERRING] NORMAL: Destination reached; releasing box
[0043] [2026-11-11 02:38:54.858] [RobotController] [DROPPING] STATE_CHANGE: Robot state changed | TRANSFERRING -> DROPPING | Reason: Robot reached the destination
[0044] [2026-11-11 02:38:54.897] [RobotController] [DROPPING] NORMAL: Box successfully dropped
[0045] [2026-11-11 02:38:54.897] [RobotController] [WAITING] STATE_CHANGE: Robot state changed | DROPPING -> WAITING | Reason: Box released successfully
[0046] [2026-11-11 02:38:54.897] [RobotController] [WAITING] NORMAL: Waiting for next task
[0047] [2026-11-11 02:38:54.929] [SystemController] [WAITING] STATE_CHANGE: System state changed | PROCESSING -> WAITING | Reason: Box processing completed
[0048] [2026-11-11 02:39:15.743] [SystemController] [WAITING] ERROR: Box waiting timeout
[0049] [2026-11-11 02:39:15.743] [SystemController] [ERROR] STATE_CHANGE: System state changed | WAITING -> ERROR | Reason: Box waiting timeout
[0050] [2026-11-11 02:39:15.743] [RobotController] [WAITING] COMMAND: Stop command received
[0051] [2026-11-11 02:39:15.743] [RobotController] [STOPPED] STATE_CHANGE: Robot state changed | WAITING -> STOPPED | Reason: Stop command
[0052] [2026-11-11 02:39:15.743] [ConveyorController] [STOPPED] COMMAND: Conveyor stopped
[0053] [2026-11-11 02:39:15.743] [ConveyorController] [STOPPED] STATE_CHANGE: Conveyor state changed | RUNNING -> STOPPED | Reason: Stop command
[main.py]: Press CTRL + C to reset and start robot.
^C[main.py]: Reset signal received.
[0054] [2026-11-11 02:39:45.207] [SystemController] [ERROR] COMMAND: Reset command received
[0055] [2026-11-11 02:39:45.207] [RobotController] [STOPPED] COMMAND: Reset command received
[0056] [2026-11-11 02:39:45.209] [SystemController] [STOPPED] STATE_CHANGE: System state changed | ERROR -> STOPPED | Reason: System reset
[0057] [2026-11-11 02:39:45.209] [SystemController] [STOPPED] COMMAND: Start command received
[0058] [2026-11-11 02:39:45.209] [RobotController] [STOPPED] COMMAND: Start command received
```

```
[0059] [2026-11-11 02:39:45.209] [RobotController] [WAITING] STATE_CHANGE: Robot state changed | STOPPED -> WAITING | Reason: Start command
[0060] [2026-11-11 02:39:45.209] [ConveyorController] [RUNNING] COMMAND: Conveyor started
[0061] [2026-11-11 02:39:45.209] [ConveyorController] [RUNNING] STATE_CHANGE: Conveyor state changed | STOPPED -> RUNNING | Reason: Start command
[0062] [2026-11-11 02:39:45.210] [SystemController] [WAITING] STATE_CHANGE: System state changed | STOPPED -> WAITING | Reason: System started
[0063] [2026-11-11 02:39:48.247] [RobotController] [WAITING] COMMAND: Box processing task received
[0064] [2026-11-11 02:39:48.248] [RobotController] [SEARCHING] STATE_CHANGE: Robot state changed | WAITING -> SEARCHING | Reason: Task started by system controller
[0065] [2026-11-11 02:39:48.248] [ConveyorController] [STOPPED] COMMAND: Conveyor stopped
[0066] [2026-11-11 02:39:48.248] [ConveyorController] [STOPPED] STATE_CHANGE: Conveyor state changed | RUNNING -> STOPPED | Reason: Stop command
[0067] [2026-11-11 02:39:48.248] [SystemController] [PROCESSING] STATE_CHANGE: System state changed | WAITING -> PROCESSING | Reason: Box processing task started
[0068] [2026-11-11 02:39:48.475] [RobotController] [GRABBING] STATE_CHANGE: Robot state changed | SEARCHING -> GRABBING | Reason: Robot aimed at box, grabbing state activated
[0069] [2026-11-11 02:39:48.475] [RobotController] [GRABBING] NORMAL: Attempting to grab box
[0070] [2026-11-11 02:39:48.493] [ConveyorController] [RUNNING] COMMAND: Conveyor started
[0071] [2026-11-11 02:39:48.493] [ConveyorController] [RUNNING] STATE_CHANGE: Conveyor state changed | STOPPED -> RUNNING | Reason: Start command
[0072] [2026-11-11 02:39:49.785] [RobotController] [GRABBING] NORMAL: Box successfully grabbed
[0073] [2026-11-11 02:39:49.786] [RobotController] [LIFTING] STATE_CHANGE: Robot state changed | GRABBING -> LIFTING | Reason: Box successfully grabbed
[0074] [2026-11-11 02:39:49.786] [RobotController] [LIFTING] NORMAL: Lifting box
[0075] [2026-11-11 02:39:50.109] [RobotController] [LIFTING] NORMAL: Box lifted successfully
[0076] [2026-11-11 02:39:50.111] [RobotController] [TRANSFERRING] STATE_CHANGE: Robot state changed | LIFTING -> TRANSFERRING | Reason: Box lifted successfully
[0077] [2026-11-11 02:39:50.111] [RobotController] [TRANSFERRING] NORMAL: Transferring box to destination
[0078] [2026-11-11 02:39:50.647] [RobotController] [TRANSFERRING] NORMAL: Destination reached; releasing box
[0079] [2026-11-11 02:39:50.647] [RobotController] [DROPPING] STATE_CHANGE: Robot state changed | TRANSFERRING -> DROPPING | Reason: Robot reached the destination
[0080] [2026-11-11 02:39:50.684] [RobotController] [DROPPING] NORMAL: Box successfully dropped
[0081] [2026-11-11 02:39:50.684] [RobotController] [WAITING] STATE_CHANGE: Robot state changed | DROPPING -> WAITING | Reason: Box released successfully
[0082] [2026-11-11 02:39:50.684] [RobotController] [WAITING] NORMAL: Waiting for next task
[0083] [2026-11-11 02:39:50.723] [SystemController] [WAITING] STATE_CHANGE: System state changed | PROCESSING -> WAITING | Reason: Box processing completed
^C
[main.py]: Stopping simulation...
[0084] [2026-11-11 02:39:53.595] [RobotController] [WAITING] SYSTEM: Robot controller cleanup started
[0085] [2026-11-11 02:39:53.599] [RobotController] [WAITING] SYSTEM: Robot controller cleanup completed
[main.py]: System cleanup completed.
[main.py]: Simulation stopped.
(.venv)
[nix-shell:~/Documents/5sem/robots/cw6]$
```

Промисловий протокол (теоретична реалізація):

Для організації обміну даними між компонентами робототехнічного комплексу запропоновано використання промислового протоколу OPC UA. Він забезпечує передавання команд керування, параметрів роботи, поточних станів і кодів помилок між системним контролером та контролерами робота і конвеєрів.

Обмін даними пропонується організувати між *SystemController, RobotController ma ConveyorController.* Системний контролер передаватиме команди запуску, зупинки, скидання помилок і початку обробки об'єкта. Також передбачено обмін параметрами (швидкість конвеєра, таймаути), станами автоматів (WAITING, PROCESSING, ERROR тощо) та повідомленнями про помилки.

Кожній команді пропонується присвоювати унікальний ідентифікатор command_id. Після отримання команди контролер надсилатиме підтвердження прийняття (ACK), а після завершення операції — підтвердження виконання (DONE) або повідомлення про помилку (ERROR) з відповідним кодом. Це дозволить відстежувати виконання окремих операцій.

Для контролю з'єднання передбачено механізм heartbeat — періодичне оновлення контрольного значення кожним контролером. Якщо протягом встановленого часу значення не оновлюється, система фіксуватиме втрату зв'язку та переходитиме в аварійний стан. Виконавчі контролери також повинні забезпечувати безпечну зупинку в разі втрати зв'язку із системним контролером.

Порядок запуску компонентів і налаштування з’єднань:

1) Відкрити сцену CoppeliaSim із запущеним ZMQ Remote API server та потрібними об’єктами: /IRB140, /Box1, /highTable, /conveyor1, /conveyor2 і suctionPad з його датчиком та dummy-об’єктами.

2) Запустити main.py. RemoteAPIClient під’єднується до localhost:23000; адреса й порт клієнта мають відповідати налаштуванням сервера.

3) main.py отримує інтерфейси sim і simIK, створює SystemController (той створює RobotController і ConveyorController), викликає start(), вмикає покроковий режим і запускає симуляцію.

4) У циклі main.py виконує SystemController.update(), потім client.step(). Після reset система лишається в STOPPED і для відновлення потребує окремої команди start().

Інтерфейси обміну (фактичні):

• main.py ↔ CoppeliaSim: ZeroMQ Remote API через RemoteAPIClient; запити до sim керують симуляцією та об’єктами, simIK виконує IK.

• SystemController ↔ дочірні контролери: виклики start(), start_task(), update(), stop() і reset(); стани та результати перевірок передаються через атрибути Python-об’єктів.

• Контролери ↔ сцена: об’єкти отримуються за шляхами CoppeliaSim; зіткнення Box1 із highTable перевіряється через sim.checkCollision, а захоплення — за батьківським об’єктом suction cup dummy.

• ConveyorController передає швидкість обох конвеєрів у customData.__ctrl__ через sim.packTable і sim.setBufferProperty: velocity при роботі, 0 при зупинці.

• Кожен контролер записує свої події через окремий EventLogger; логування призначене для діагностики й не є каналом керування.

Висновки:

У ході виконання лабораторної роботи було розроблено розподілену за функціями систему керування робототехнічним комплексом у середовищі CoppeliaSim за допомогою Python. Реалізовано окремі контролери для керування роботом ABB IRB140 та конвеєрами, а також системний контролер для координації їхньої роботи. За допомогою автоматів станів забезпечено виявлення об'єктів, зупинку та відновлення руху конвеєрів, захоплення, переміщення й розміщення об'єктів роботом. Для керування маніпулятором використано інверсну кінематику (IK), а для захоплення об'єктів — вакуумну присоску. Також реалізовано обробку помилок,

механізм скидання та журналювання подій. У результаті створено систему, що забезпечує узгоджену автоматичну роботу компонентів робототехнічного комплексу та виконання циклу переміщення об'єктів.